WhoPlay — Guia simples do Login

Nov 5, 2026 · @Sebastião Cardoso

Visão geral

No fim deste guia tens uma app Flutter com 2 ecrãs (Login e Onboarding) em que o login é verificado numa base de dados MySQL, através de uma pequena API em Node.js. É a versão mínima do projeto: só login, uma tabela, uma rota.

O guia tem 7 partes, e cada uma funciona sozinha antes de passares à seguinte:

1. **Ecrã de Login** — só Flutter, sem base de dados.
2. **Ecrã de Onboarding** — escolher desportos; o botão Entrar passa a abrir este ecrã.
3. **Login com base de dados** — MySQL no XAMPP, API em Node.js e a app a falar com a API.
4. **Registo** — o botão Criar conta abre um ecrã novo que grava o utilizador na base de dados.
5. **Onboarding completo** — 3 passos (desportos, nível, disponibilidade) gravados na base de dados.
6. **Perfil** — mostra os dados guardados, com Editar perfil e Terminar sessão.
7. **Visual dos mockups** — as letras, cores e peças dos mockups no Onboarding e no Perfil.

Programas de que precisas

Programa	Para quê	Onde arranjar
Flutter SDK	Criar e correr a app	flutter.dev (já tens em `C:\src\flutter`)
VS Code + extensão Flutter	Escrever o código	code.visualstudio.com
Google Chrome	Ver a app a correr	já tens
XAMPP	MySQL + phpMyAdmin (a base de dados)	apachefriends.org (já tens em `C:\xampp`)
Node.js, versão LTS	Correr a API	nodejs.org (já tens, v24)

Antes de começar: o terminal

Todos os comandos deste guia são para o **Git Bash**, o terminal que mostra `MINGW64` (vem com o Git para Windows e também pode ser o terminal do VS Code).

Copia o código deste guia online, não do PDF. O PDF parte as linhas compridas para caberem na página, e um texto entre aspas partido ao meio dá erro.

Pôr o `flutter` a funcionar no Git Bash

O Flutter está em `C:\src\flutter`, mas o Git Bash não o encontra sozinho (dá `flutter: command not found`). Corre isto uma vez:

```
echo 'export PATH="$PATH:/c/src/flutter/bin"' >> ~/.bashrc
source ~/.bashrc
```

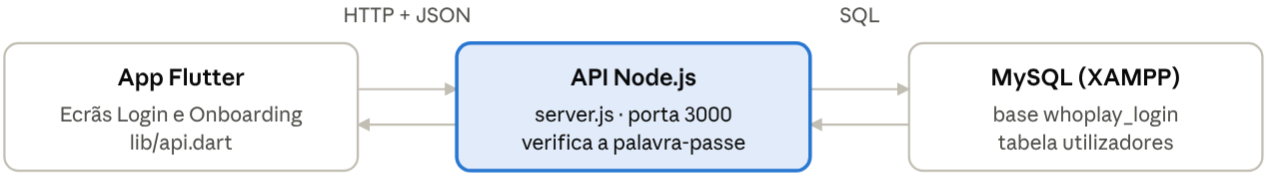
Na próxima vez que abrires o Git Bash pode aparecer `WARNING: Found ~/.bashrc but no ~/.bash_profile…`. É normal e só aparece uma vez.

Para confirmar, corre `flutter --version` e `node -v`: os dois têm de mostrar um número de versão.

Como as peças falam umas com as outras

A app nunca fala diretamente com o MySQL: pede à API, e só a API fala com a base de dados. Assim a palavra-passe da base de dados nunca vai dentro da app.

A app só fala com a API; só a API fala com o MySQL



arquitetura · 3 peças

A app envia o email e a palavra-passe à API; a API confirma-os no MySQL e responde com o nome ou com um erro.

Parte 1 — Ecrã de Login

Nesta parte crias o projeto e o ecrã de Login. O botão Entrar ainda só mostra uma mensagem.

Passo 1 — Criar o projeto

O guia usa pastas novas em **Documentos**, ao lado do projeto `whoplay`, para não mexer na app completa (os 10 ecrãs) que já lá está:

Pasta	O que tem
`C:\Users\cardoso\Documents\whoplay`	O projeto completo (não mexer)
`C:\Users\cardoso\Documents\whoplay_login`	A app deste guia (Partes 1, 2, 3.3, 4, 5, 6 e 7)
`C:\Users\cardoso\Documents\whoplay_login_api`	A API deste guia (Partes 3.2, 4, 5 e 6)

No Git Bash:

```
cd ~/Documents
flutter create whoplay_login
cd whoplay_login
code .
```

O `code .` abre a pasta no VS Code. Todo o código deste guia fica dentro da pasta `lib/`.

Passo 2 — As cores (`lib/cores.dart`)

Cria o ficheiro `lib/cores.dart`. Fica com as cores dos mockups num só sítio:

```
import 'package:flutter/material.dart';

// Cores do WhoPlay (as mesmas dos mockups).
const verde = Color(0xFF1F7A4D);
const floresta = Color(0xFF123D29);
const lima = Color(0xFFC9F25C);
const fundo = Color(0xFFF3F5F0);
```

Passo 3 — O ecrã (`lib/login_screen.dart`)

Cria o ficheiro `lib/login_screen.dart`. O ecrã tem 2 zonas: um cabeçalho verde com o nome e o slogan, e o formulário por baixo.

```
import 'package:flutter/material.dart';

import 'cores.dart';

class LoginScreen extends StatefulWidget {
```

```
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  // Guardam o que o utilizador escreve nos campos
  final _email = TextEditingController();
  final _password = TextEditingController();

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  // Mostra uma mensagem no fundo do ecrã
  void _mostrar(String mensagem) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:
Text(mensagem)));
  }

  void _entrar() {
    _mostrar('Olá, ${_email.text}!');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: fundo,
      body: ListView(
        children: [
          // Cabeçalho verde com o nome e o slogan
          Container(
            color: floresta,
            padding: const EdgeInsets.fromLTRB(28, 80, 28, 40),
            child: const Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  'WhoPlay',
                  style: TextStyle(color: lima, fontSize: 40, fontWeight:
FontWeight.bold),
```

```
                ),
                SizedBox(height: 12),
                Text(
                  'Encontra pessoas. Marca o jogo. Vai jogar.',
                  style: TextStyle(color: Colors.white, fontSize: 18),
                ),
              ],
            ),
          ),
          // Formulário
          Padding(
            padding: const EdgeInsets.all(28),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                const Text('Entrar', style: TextStyle(fontSize: 28,
fontWeight: FontWeight.bold)),
                const SizedBox(height: 20),
                TextField(
                  controller: _email,
                  keyboardType: TextInputType.emailAddress,
                  decoration: const InputDecoration(
                    labelText: 'Email',
                    filled: true,
                    fillColor: Colors.white,
                    border: OutlineInputBorder(),
                  ),
                ),
                const SizedBox(height: 16),
                TextField(
                  controller: _password,
                  obscureText: true, // esconde a palavra-passe com pontos
                  decoration: const InputDecoration(
                    labelText: 'Palavra-passe',
                    filled: true,
                    fillColor: Colors.white,
                    border: OutlineInputBorder(),
                  ),
                ),
                const SizedBox(height: 24),
                FilledButton(
                  onPressed: _entrar,
                  style: FilledButton.styleFrom(
                    backgroundColor: verde,
                    minimumSize: const Size.fromHeight(52),
                  ),
```

```
                  child: const Text('Entrar'),
                ),
                const SizedBox(height: 32),
                const Text('Ainda não tens conta?', textAlign:
TextAlign.center),
                const SizedBox(height: 8),
                OutlinedButton(
                  onPressed: () => _mostrar('O registo fica para a próxima
versão.'),
                  style: OutlinedButton.styleFrom(
                    foregroundColor: verde,
                    minimumSize: const Size.fromHeight(52),
                  ),
                  child: const Text('Criar conta'),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

Passo 4 — Abrir a app no Login (`lib/main.dart`)

Apaga tudo o que está em `lib/main.dart` e põe isto:

```
import 'package:flutter/material.dart';

import 'login_screen.dart';

void main() {
  runApp(const WhoPlayApp());
}

class WhoPlayApp extends StatelessWidget {
  const WhoPlayApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'WhoPlay',
      debugShowCheckedModeBanner: false,
```

```
      home: LoginScreen(),
    );
  }
}
```

Apaga também a pasta `test/`: o teste que vem com o `flutter create` é da app de exemplo e dá o erro `The name 'MyApp' isn't a class`. Na pasta `whoplay_login`:

```
rm -r test
```

Passo 5 — Correr

```
flutter run -d chrome
```

O que deves ver: o cabeçalho verde-escuro com "WhoPlay", os campos Email e Palavra-passe, e os botões Entrar e Criar conta. Escreve um email e carrega em Entrar: aparece "Olá, …!" no fundo do ecrã.

Parte 2 — Ecrã de Onboarding

Nesta parte crias o ecrã "Que desportos queres jogar?" e o botão Entrar passa a abri-lo.

Passo 1 — O ecrã (`lib/onboarding_screen.dart`)

Cria o ficheiro `lib/onboarding_screen.dart`. Tem a barra "Passo 1 de 3", uma grelha com os 8 desportos e o botão Continuar. Tocar num desporto marca-o (fica verde-lima); tocar outra vez desmarca-o.

```
import 'package:flutter/material.dart';

import 'cores.dart';

const desportos = ['Padel', 'Ténis', 'Futebol', 'BJJ', 'Basquetebol',
'Voleibol', 'Running', 'Ginásio'];

class OnboardingScreen extends StatefulWidget {
  const OnboardingScreen({super.key});

  @override
  State<OnboardingScreen> createState() => _OnboardingScreenState();
}

class _OnboardingScreenState extends State<OnboardingScreen> {
  // Os desportos que o utilizador escolheu
```

```
  final _escolhidos = <String>{};

  void _tocar(String desporto) {
    setState(() {
      if (_escolhidos.contains(desporto)) {
        _escolhidos.remove(desporto);
      } else {
        _escolhidos.add(desporto);
      }
    });
  }

  void _continuar() {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Escolheste: ${_escolhidos.join(', ')}')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: fundo,
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Text('Passo 1 de 3'),
              const SizedBox(height: 8),
              const LinearProgressIndicator(value: 1 / 3, color: verde,
minHeight: 6),
              const SizedBox(height: 24),
              const Text(
                'Que desportos queres jogar?',
                style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 8),
              const Text('Escolhe um ou mais. Podes mudar isto no teu
perfil.'),
              const SizedBox(height: 24),
              Expanded(
                child: GridView.count(
                  crossAxisCount: 2,
                  mainAxisSpacing: 12,
                  crossAxisSpacing: 12,
```

```
                childAspectRatio: 2.2,
                children: [
                  for (final desporto in desportos)
                    CartaoDesporto(
                      nome: desporto,
                      escolhido: _escolhidos.contains(desporto),
                      onTap: () => _tocar(desporto),
                    ),
                ],
              ),
            ),
            Text('${_escolhidos.length} escolhidos', textAlign:
TextAlign.center),
            const SizedBox(height: 8),
            FilledButton(
              // Sem nenhum desporto escolhido, o botão fica desligado
              onPressed: _escolhidos.isEmpty ? null : _continuar,
              style: FilledButton.styleFrom(
                backgroundColor: verde,
                minimumSize: const Size.fromHeight(52),
              ),
              child: const Text('Continuar'),
            ),
          ],
        ),
      ),
    ),
  );
  }
}

// Um cartão da grelha: branco quando não está escolhido, verde-lima quando
está
class CartaoDesporto extends StatelessWidget {
  const CartaoDesporto({super.key, required this.nome, required
this.escolhido, required this.onTap});

  final String nome;
  final bool escolhido;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: escolhido ? lima : Colors.white,
      shape: RoundedRectangleBorder(
```

```
        borderRadius: BorderRadius.circular(16),
        side: BorderSide(color: escolhido ? Colors.black : Colors.black26,
width: escolhido ? 2 : 1),
      ),
      child: InkWell(
        borderRadius: BorderRadius.circular(16),
        onTap: onTap,
        child: Center(
          child: Text(nome, style: const TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
        ),
      ),
    );
  }
}
```

Passo 2 — Ligar o Entrar ao Onboarding

Em `lib/login_screen.dart`, junta este import no topo, por baixo de `import 'cores.dart';`:

```
import 'onboarding_screen.dart';
```

E troca a função `_entrar` por esta:

```
  void _entrar() {
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => const OnboardingScreen()),
    );
  }
```

`pushReplacement` troca o Login pelo Onboarding: o botão "voltar" já não regressa ao Login.

Passo 3 — Experimentar

No terminal onde a app está a correr, carrega em **R** (maiúsculo) para reiniciar.

O que deves ver: carregas em Entrar e abre o Onboarding. O Continuar está cinzento até escolheres um desporto; depois mostra "Escolheste: Padel, …".

Parte 3.1 — Base de dados no XAMPP

Crias a base `whoplay_login` com uma só tabela, `utilizadores`, e um utilizador de teste. O nome é diferente de `whoplay` para não mexer na base do projeto completo.

Passo 1 — Ligar o MySQL

1. Abre o **XAMPP Control Panel**.
2. Carrega em **Start** na linha **Apache** e na linha **MySQL**. As duas ficam a verde.
3. Na linha MySQL, carrega em **Admin**. Abre o phpMyAdmin no navegador (`http://localhost/phpmyadmin`).

O Apache só é preciso para o phpMyAdmin; a API não o usa.

Passo 2 — Criar a base e a tabela

No phpMyAdmin, abre o separador **SQL** (no topo), cola isto e carrega em **Executar**:

```
CREATE DATABASE IF NOT EXISTS whoplay_login
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE whoplay_login;

CREATE TABLE utilizadores (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  nome          VARCHAR(100) NOT NULL,
  email         VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL
);

-- Utilizador de teste. A palavra-passe é 12345678 (guardada encriptada com
bcrypt).
INSERT INTO utilizadores (nome, email, password_hash) VALUES
  ('Sebastião', 'sebastiao@email.com',
'$2b$10$pmW7t7Clw0vlmN2AmEkgd.CI3VDEi4DeFultw7XPSs8aZHl5Szzw2');
```

O que deves ver: à esquerda aparece a base `whoplay_login`. Clica em **utilizadores** debaixo dela: tem de ter 1 linha, a do Sebastião.

Se a tabela estiver vazia (e mais tarde o login der sempre "Email ou palavra-passe errados."), o `INSERT` não correu. Clica em **whoplay_login** (não em `whoplay`, que também tem uma tabela `utilizadores`), abre o separador **SQL** e corre só esta linha:

```
INSERT INTO utilizadores (nome, email, password_hash) VALUES ('Sebastião',
'sebastiao@email.com',
'$2b$10$pmW7t7Clw0vlmN2AmEkgd.CI3VDEi4DeFultw7XPSs8aZHl5Szzw2');
```

Porque é que a palavra-passe aparece estranha?

A base nunca guarda a palavra-passe verdadeira: guarda um *hash* (a `$2b$10$…`), feito com bcrypt. Se alguém roubar a tabela, não fica a saber as palavras-passe. No login, a API compara o que o utilizador escreveu com esse hash.

Para criar outro utilizador, gera o hash da palavra-passe dele (depois de fazeres a Parte 3.2, dentro da pasta da API):

```
node -e "console.log(require('bcryptjs').hashSync('a-palavra-passe', 10))"
```

E usa o texto que aparece num novo `INSERT INTO utilizadores …`.

Parte 3.2 — API em Node.js

A API é um único ficheiro, `server.js`, com uma rota: `POST /login`. Recebe o email e a palavra-passe, procura o utilizador no MySQL e responde com o nome dele ou com um erro.

Passo 1 — Criar a pasta e instalar os pacotes

Num Git Bash **novo**, cria a pasta da API em Documentos, ao lado de `whoplay_login` (não dentro dela):

```
cd ~/Documents
mkdir whoplay_login_api
cd whoplay_login_api
npm init -y
npm install express cors mysql2 bcryptjs
```

O que deves ver: no fim, `added … packages` e `found 0 vulnerabilities`.

Pacote	Para quê
`express`	Criar o servidor e a rota `/login`
`cors`	Deixar a app que corre no Chrome falar com a API
`mysql2`	Ligar ao MySQL do XAMPP
`bcryptjs`	Comparar a palavra-passe com o hash guardado

Passo 2 — O servidor (`server.js`)

Cria o ficheiro `server.js` **dentro de** **`whoplay_login_api`** , ao lado do `package.json` e da pasta `node_modules` . Se o puseres na pasta da app (`whoplay_login`), dá o erro `Cannot find module 'express'` .

```
const express = require('express');
const cors = require('cors');
const mysql = require('mysql2/promise');
const bcrypt = require('bcryptjs');

const app = express();
app.use(cors()); // deixa a app no Chrome falar com a API
app.use(express.json()); // lê o corpo dos pedidos em JSON

// Ligação ao MySQL do XAMPP (utilizador root, sem palavra-passe)
const db = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'whoplay_login',
});

// POST /login  →  { "email": "...", "password": "..." }
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ erro: 'Preenche o email e a palavra-passe.'
});
  }

  const [linhas] = await db.query(
    'SELECT id, nome, password_hash FROM utilizadores WHERE email = ?',
    [email],
  );
```

```
  const utilizador = linhas[0];

  if (!utilizador || !(await bcrypt.compare(password,
utilizador.password_hash))) {
    return res.status(401).json({ erro: 'Email ou palavra-passe errados.' });
  }

  res.json({ id: utilizador.id, nome: utilizador.nome });
});

app.listen(3000, () => {
  console.log('API a correr em http://localhost:3000');
});
```

O `?` na consulta SQL é importante: o `mysql2` põe lá o email em segurança. Nunca juntes o email ao texto do SQL à mão (isso abre a porta a *SQL injection*).

Quando o email não existe e quando a palavra-passe está errada, a resposta é a mesma. Assim ninguém consegue descobrir que emails têm conta.

Passo 3 — Ligar a API

```
cd ~/Documents/whoplay_login_api
node server.js
```

O que deves ver: `API a correr em http://localhost:3000`. Deixa este terminal aberto; fechar o terminal desliga a API.

Passo 4 — Testar a API sem a app

Noutro Git Bash, pede um login. É uma linha só:

```
curl -X POST http://localhost:3000/login -H "Content-Type: application/json"
-d '{"email":"sebastiao@email.com","password":"12345678"}'
```

O que deves ver: `{"id":1,"nome":"Sebastião"}`. Troca `12345678` por outra palavra-passe e repete: aparece `{"erro":"Email ou palavra-passe errados."}`.

Se aparecer o erro com a palavra-passe certa, a tabela está vazia: volta à Parte 3.1 e corre o `INSERT`.

Parte 3.3 — Ligar o Flutter à API

O botão Entrar passa a pedir o login à API. Só abre o Onboarding se o email e a palavra-passe estiverem certos.

Passo 1 — Instalar o pacote `http`

No terminal da app Flutter (pasta `whoplay_login`):

```
flutter pub add http
```

Passo 2 — A ligação à API (`lib/api.dart`)

Cria o ficheiro `lib/api.dart`. É o único sítio da app que sabe o endereço da API:

```
import 'dart:convert';

import 'package:http/http.dart' as http;

// No Chrome: localhost. No emulador Android: 'http://10.0.2.2:3000'
const apiUrl = 'http://localhost:3000';

/// Erro com a mensagem que a API devolveu (ex.: "Email ou palavra-passe
errados.").
class LoginErro implements Exception {
  LoginErro(this.mensagem);

  final String mensagem;
}

/// Pede o login à API. Se estiver certo, devolve o nome do utilizador.
Future<String> login(String email, String password) async {
  final resposta = await http.post(
    Uri.parse('$apiUrl/login'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'email': email, 'password': password}),
  );
  final dados = jsonDecode(resposta.body);
  if (resposta.statusCode != 200) {
    throw LoginErro(dados['erro']);
  }
  return dados['nome'];
}
```

Passo 3 — O botão Entrar usa a API

Em `lib/login_screen.dart` fazes 4 mudanças.

a) Junta este import no topo:

```
import 'api.dart';
```

b) Por baixo de `final _password = TextEditingController();`, junta uma variável que diz se o pedido está a decorrer:

```
  bool _aEntrar = false;
```

c) Troca a função `_entrar` por esta:

```
  Future<void> _entrar() async {
    setState(() => _aEntrar = true);
    try {
      final nome = await login(_email.text.trim(), _password.text);
      _mostrar('Bem-vindo, $nome!');
      if (!mounted) return;
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (_) => const OnboardingScreen()),
      );
    } on LoginErro catch (erro) {
      _mostrar(erro.mensagem); // ex.: "Email ou palavra-passe errados."
    } catch (_) {
      _mostrar('Não foi possível ligar à API. Ela está a correr?');
    } finally {
      if (mounted) setState(() => _aEntrar = false);
    }
  }
```

d) No botão Entrar, troca as linhas `onPressed` e `child` por estas. Enquanto espera pela API, o botão fica desligado e diz "A entrar…":

```
                  onPressed: _aEntrar ? null : _entrar,
```

```
                  child: Text(_aEntrar ? 'A entrar…' : 'Entrar'),
```

Passo 4 — Correr outra vez

O pacote `http` só entra com a app reiniciada de raiz. No terminal da app, carrega em **q** para parar e volta a correr:

```
flutter run -d chrome
```

Testar tudo e problemas comuns

Para o login funcionar têm de estar 3 coisas ligadas ao mesmo tempo: o **MySQL** no XAMPP, a **API** (`node server.js` , num terminal) e a **app** (`flutter run -d chrome` , noutro terminal).

Os 3 testes

- [ ] Email `sebastiao@email.com` e palavra-passe `12345678` → aparece "Bem-vindo, Sebastião!" e abre o Onboarding.
- [ ] Palavra-passe errada → aparece "Email ou palavra-passe errados." e fica no Login.
- [ ] Desliga a API (Ctrl+C no terminal dela) e carrega em Entrar → aparece "Não foi possível ligar à API. Ela está a correr?"

Problemas comuns

O que aparece	Porquê	Como resolver
`flutter: command not found`	O Git Bash não sabe onde está o Flutter	Os 2 comandos de "Pôr o `flutter` a funcionar no Git Bash" (Visão geral)
`Unterminated string literal` ou linhas a vermelho no VS Code	Código copiado do PDF, com linhas partidas a meio	Copiar o ficheiro inteiro outra vez, a partir do guia online
`The name 'MyApp' isn't a class`	A pasta `test/` do `flutter create` ainda existe	`rm -r test` na pasta `whoplay_login`
Aviso amarelo `Unused import: 'onboarding_screen.dart'`	Falta trocar a função `_entrar` (Parte 2, Passo 2)	Fazer esse passo; o aviso desaparece
`Cannot find module 'express'`	O `server.js` está fora da pasta da API, ou os pacotes não foram instalados	Pôr o `server.js` em `~/Documents/whoplay_login_api` e correr lá `npm install express cors mysql2 bcryptjs`
`Invoke-RestMethod: command not found`	É um comando do PowerShell	No Git Bash, usar o `curl` da Parte 3.2
`npm.ps1 não está assinado digitalmente`	Estás no PowerShell, que bloqueia o `npm`	Usar o Git Bash, ou escrever `npm.cmd` em vez de `npm`

O que aparece	Porquê	Como resolver
"Email ou palavra-passe errados." com a palavra-passe certa	A tabela `utilizadores` está vazia (o `INSERT` não correu)	Correr o `INSERT` da Parte 3.1 com a base `whoplay_login` selecionada
Na app: "Não foi possível ligar à API"	A API não está a correr	`node server.js` em `~/Documents/whoplay_login_api`
`AggregateError` ou `ECONNREFUSED` no terminal da API	O MySQL está desligado	Start no MySQL do XAMPP
`Unknown database 'whoplay_login'`	A base não foi criada	Correr o SQL da Parte 3.1 no phpMyAdmin
`Error: listen EADDRINUSE ... 3000`	Já há outra API na porta 3000 (por exemplo, a do projeto completo)	Fechar o outro terminal, ou mudar `3000` no `server.js` e no `api.dart`
No emulador Android nunca liga	No emulador, `localhost` é o próprio telemóvel	Em `api.dart`, usar `http://10.0.2.2:3000`
O MySQL do XAMPP não fica verde	A porta 3306 está ocupada por outro MySQL	Fechar o outro MySQL (Serviços do Windows) e carregar outra vez em Start

Parte 4 — Registo

O botão Criar conta passa a abrir um ecrã com Nome, Email e Palavra-passe. A API encripta a palavra-passe, grava o utilizador na tabela `utilizadores` e a app abre o Onboarding. Não é preciso mexer na base de dados: a tabela da Parte 3.1 já serve.

Passo 1 — A rota `POST /registo` (`server.js`)

Em `~/Documents/whoplay_login_api/server.js`, cola isto **antes** de `app.listen(3000, ...)`:

```
// POST /registo  →  { "nome": "...", "email": "...", "password": "..." }
app.post('/registo', async (req, res) => {
  const { nome, email, password } = req.body;
  if (!nome || !email || !password) {
```

```
    return res.status(400).json({ erro: 'Preenche o nome, o email e a
palavra-passe.' });
  }
  if (password.length < 8) {
    return res.status(400).json({ erro: 'A palavra-passe tem de ter pelo
menos 8 caracteres.' });
  }

  // Nunca guardar a palavra-passe: guarda-se só o hash
  const hash = await bcrypt.hash(password, 10);

  try {
    const [resultado] = await db.query(
      'INSERT INTO utilizadores (nome, email, password_hash) VALUES (?, ?,
?)',
      [nome, email, hash],
    );
    res.status(201).json({ id: resultado.insertId, nome });
  } catch (erro) {
    // A coluna email é UNIQUE: o MySQL recusa um email repetido
    if (erro.code === 'ER_DUP_ENTRY') {
      return res.status(409).json({ erro: 'Já existe uma conta com este
email.' });
    }
    throw erro;
  }
});
```

O `bcrypt.hash` faz o mesmo que o comando `node -e …` da Parte 3.1, mas automático. O email repetido é apanhado pela regra `UNIQUE` da tabela, por isso a API não precisa de procurar o email antes.

No terminal da API, carrega em **Ctrl+C** e volta a ligar:

```
node server.js
```

Passo 2 — Testar a rota com `curl`

Noutro Git Bash (uma linha só):

```
curl -X POST http://localhost:3000/registo -H "Content-Type:
application/json" -d
'{"nome":"Ana","email":"ana@email.com","password":"segredo123"}'
```

Resposta	Quando
`{"id":2,"nome":"Ana"}`	Da primeira vez: a conta foi criada (o id pode ser outro número)
`{"erro":"Já existe uma conta com este email."}`	Se repetires o mesmo comando
`{"erro":"A palavra-passe tem de ter pelo menos 8 caracteres."}`	Se puseres uma palavra-passe curta

No phpMyAdmin, a tabela `utilizadores` passa a ter a Ana, com a palavra-passe guardada como hash (`$2b$10$…`). A nova conta já entra no login:

```
curl -X POST http://localhost:3000/login -H "Content-Type: application/json"
-d '{"email":"ana@email.com","password":"segredo123"}'
```

Passo 3 — A função `registo()` (`lib/api.dart`)

Na app, cola isto no **fim** de `lib/api.dart` . Usa a mesma classe `LoginErro` para as mensagens de erro da API:

```
/// Cria a conta na API. Se correr bem, devolve o nome do utilizador.
Future<String> registo(String nome, String email, String password) async {
  final resposta = await http.post(
    Uri.parse('$apiUrl/registo'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'nome': nome, 'email': email, 'password': password}),
  );
  final dados = jsonDecode(resposta.body);
  if (resposta.statusCode != 201) {
    throw LoginErro(dados['erro']);
  }
  return dados['nome'];
}
```

Passo 4 — O ecrã Criar conta (`lib/register_screen.dart`)

Cria o ficheiro `lib/register_screen.dart` :

```
import 'package:flutter/material.dart';

import 'api.dart';
import 'cores.dart';
import 'onboarding_screen.dart';
```

```
class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _nome = TextEditingController();
  final _email = TextEditingController();
  final _password = TextEditingController();
  bool _aCriar = false;

  @override
  void dispose() {
    _nome.dispose();
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  void _mostrar(String mensagem) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(mensagem)),
    );
  }

  Future<void> _criarConta() async {
    setState(() => _aCriar = true);
    try {
      final nome = await registo(
        _nome.text.trim(),
        _email.text.trim(),
        _password.text,
      );
      _mostrar('Conta criada. Bem-vindo, $nome!');
      if (!mounted) return;
      // Abre o Onboarding e apaga o histórico (o "voltar" já não volta aqui)
      Navigator.pushAndRemoveUntil(
        context,
        MaterialPageRoute(builder: (_) => const OnboardingScreen()),
        (_) => false,
      );
    } on LoginErro catch (erro) {
```

```
      _mostrar(erro.mensagem); // ex.: "Já existe uma conta com este email."
    } catch (_) {
      _mostrar('Não foi possível ligar à API. Ela está a correr?');
    } finally {
      if (mounted) setState(() => _aCriar = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: fundo,
      appBar: AppBar(
        backgroundColor: fundo,
        title: const Text('Criar conta'),
      ),
      body: ListView(
        padding: const EdgeInsets.all(28),
        children: [
          TextField(
            controller: _nome,
            decoration: const InputDecoration(
              labelText: 'Nome',
              filled: true,
              fillColor: Colors.white,
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 16),
          TextField(
            controller: _email,
            keyboardType: TextInputType.emailAddress,
            decoration: const InputDecoration(
              labelText: 'Email',
              filled: true,
              fillColor: Colors.white,
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 16),
          TextField(
            controller: _password,
            obscureText: true,
            decoration: const InputDecoration(
              labelText: 'Palavra-passe',
              helperText: 'Mínimo 8 caracteres',
```

```
              filled: true,
              fillColor: Colors.white,
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 24),
          FilledButton(
            onPressed: _aCriar ? null : _criarConta,
            style: FilledButton.styleFrom(
              backgroundColor: verde,
              minimumSize: const Size.fromHeight(52),
            ),
            child: Text(_aCriar ? 'A criar…' : 'Criar conta'),
          ),
        ],
      ),
    );
  }
}
```

`pushAndRemoveUntil` abre o Onboarding e apaga o Login e o Criar conta do histórico: depois de criar a conta, o "voltar" já não regressa ao formulário.

Passo 5 — Ligar o botão Criar conta (`lib/login_screen.dart`)

a) Junta este import no topo, por baixo de `import 'onboarding_screen.dart';`:

```
import 'register_screen.dart';
```

b) No botão Criar conta (o `OutlinedButton`), troca a linha `onPressed: () => _mostrar('O registo fica para a próxima versão.'),` por esta:

```
                  onPressed: () => Navigator.push(
                    context,
                    MaterialPageRoute(builder: (_) => const
RegisterScreen()),
                  ),
```

Aqui usa-se `push` , e não `pushReplacement` , para a seta de voltar do Criar conta regressar ao Login.

Passo 6 — Experimentar

Com o MySQL e a API ligados, no terminal da app carrega em **R** (maiúsculo).

- [ ] Login → Criar conta → abre o ecrã com Nome, Email e Palavra-passe; a seta volta ao Login.
- [ ] Preencher com um email novo e palavra-passe de 8 ou mais caracteres → "Conta criada. Bem-vindo, …!" e abre o Onboarding.
- [ ] Repetir com o mesmo email → "Já existe uma conta com este email." e fica no formulário.
- [ ] Palavra-passe com menos de 8 caracteres → "A palavra-passe tem de ter pelo menos 8 caracteres."
- [ ] Reiniciar a app e entrar no Login com a conta nova → "Bem-vindo, …!"

A seguir, a Parte 5 grava as escolhas do Onboarding na base de dados.

Parte 5 — Onboarding completo (3 passos + base de dados)

O Onboarding passa a ter os 3 passos do mockup — **desportos**, **nível** e **disponibilidade** — e no fim grava tudo na base de dados, ligado ao utilizador que entrou. Depois mostra um ecrã "Perfil guardado!".

Pedaço	O que muda
Base de dados	2 tabelas novas: `utilizador_desportos` e `disponibilidades`
API (`server.js`)	Rota nova `POST /onboarding`
App (`lib/api.dart`)	Guarda o `id` de quem entrou + função `guardarOnboarding()`
App (`lib/onboarding_screen.dart`)	Ficheiro novo inteiro, com os 3 passos

Passo 1 — As tabelas novas

No phpMyAdmin, clica em **whoplay_login** → separador **SQL** → cola isto → **Executar**:

```
USE whoplay_login;

-- Que desportos cada utilizador joga, e com que nível
CREATE TABLE utilizador_desportos (
  utilizador_id INT NOT NULL,
  desporto      VARCHAR(30) NOT NULL,
  nivel         VARCHAR(20) NOT NULL,
  PRIMARY KEY (utilizador_id, desporto),
  FOREIGN KEY (utilizador_id) REFERENCES utilizadores(id) ON DELETE CASCADE
);

-- Quando cada utilizador pode jogar (ex.: Sáb, Manhã)
CREATE TABLE disponibilidades (
  utilizador_id INT NOT NULL,
  dia           VARCHAR(3) NOT NULL,
  periodo       VARCHAR(10) NOT NULL,
  PRIMARY KEY (utilizador_id, dia, periodo),
  FOREIGN KEY (utilizador_id) REFERENCES utilizadores(id) ON DELETE CASCADE
);
```

O que deves ver: debaixo de `whoplay_login` passam a estar 3 tabelas: `utilizadores`, `utilizador_desportos` e `disponibilidades`.

A `FOREIGN KEY` liga cada linha a um utilizador que existe: o MySQL recusa gravar desportos de um `id` que não está em `utilizadores`. O `ON DELETE CASCADE` apaga os desportos e horários quando se apaga o utilizador.

Passo 2 — A rota `POST /onboarding` (`server.js`)

Em `~/Documents/whoplay_login_api/server.js`, cola isto **antes** de `app.listen(3000, ...)`:

```
// POST /onboarding  →  { "utilizador_id": 1, "desportos": [...],
"disponibilidade": [...] }
app.post('/onboarding', async (req, res) => {
  const { utilizador_id, desportos, disponibilidade } = req.body;
  if (!utilizador_id) {
    return res.status(400).json({ erro: 'Falta saber quem és. Entra outra
vez.' });
  }
  if (!Array.isArray(desportos) || desportos.length === 0) {
    return res.status(400).json({ erro: 'Escolhe pelo menos um desporto.' });
  }
  if (!Array.isArray(disponibilidade) || disponibilidade.length === 0) {
    return res.status(400).json({ erro: 'Escolhe pelo menos um horário.' });
  }
```

```
  // Uma transação: ou grava tudo, ou não grava nada
  const ligacao = await db.getConnection();
  try {
    await ligacao.beginTransaction();

    // Apaga o que havia antes, para se poder repetir o onboarding
    await ligacao.query('DELETE FROM utilizador_desportos WHERE utilizador_id
= ?', [utilizador_id]);
    await ligacao.query('DELETE FROM disponibilidades WHERE utilizador_id =
?', [utilizador_id]);

    for (const d of desportos) {
      await ligacao.query(
        'INSERT INTO utilizador_desportos (utilizador_id, desporto, nivel)
VALUES (?, ?, ?)',
        [utilizador_id, d.desporto, d.nivel],
      );
    }
    for (const h of disponibilidade) {
      await ligacao.query(
        'INSERT INTO disponibilidades (utilizador_id, dia, periodo) VALUES
(?, ?, ?)',
        [utilizador_id, h.dia, h.periodo],
      );
    }

    await ligacao.commit();
    res.json({ ok: true });
  } catch (erro) {
    await ligacao.rollback();
    // O utilizador_id não existe na tabela utilizadores
    if (erro.code === 'ER_NO_REFERENCED_ROW_2') {
      return res.status(404).json({ erro: 'Esse utilizador não existe. Entra
outra vez.' });
    }
    throw erro;
  } finally {
    ligacao.release();
  }
});
```

A **transação** junta várias operações numa só: se uma falhar, o `rollback` desfaz as outras, e nunca fica um perfil gravado a meio. Apagar antes de gravar permite repetir o onboarding: as escolhas novas substituem as antigas.

No terminal da API, **Ctrl+C** e volta a ligar:

```
node server.js
```

Passo 3 — Testar a rota com `curl`

Grava um perfil para o Sebastião (`id` 1). Pode ser copiado em várias linhas: o `\` no fim diz ao Git Bash que o comando continua.

```
curl -X POST http://localhost:3000/onboarding \
  -H "Content-Type: application/json" \
  -d '{"utilizador_id":1,"desportos":
[{"desporto":"Padel","nivel":"Iniciante"}],"disponibilidade":
[{"dia":"Seg","periodo":"Noite"}]}'
```

O que deves ver: `{"ok":true}`. No phpMyAdmin, `utilizador_desportos` tem a linha `1 · Padel · Iniciante` e `disponibilidades` tem `1 · Seg · Noite`.

Não uses acentos no `curl`. No Windows, o `curl` do Git Bash não envia os acentos em UTF-8, e a base grava `Interm�dio` ou `S�b`. Pela app isto não acontece. Se já tens linhas assim, entra na app com esse utilizador e refaz o Onboarding (ou usa "Editar perfil" da Parte 6): as escolhas novas substituem as estragadas.

Passo 4 — Guardar quem entrou (`lib/api.dart`)

A app tem de saber **quem** entrou para gravar os desportos no utilizador certo. O login e o registo já recebem o `id` da API; agora ficam com ele na variável `utilizadorAtual`.

Substitui o `lib/api.dart` **inteiro** por isto:

```
import 'dart:convert';

import 'package:http/http.dart' as http;

// No Chrome: localhost. No emulador Android: 'http://10.0.2.2:3000'
const apiUrl = 'http://localhost:3000';

/// O id de quem entrou. Fica guardado depois do login ou do registo.
int? utilizadorAtual;

/// Erro com a mensagem que a API devolveu (ex.: "Email ou palavra-passe
errados.").
class LoginErro implements Exception {
  LoginErro(this.mensagem);

  final String mensagem;
```

```
}

/// Pede o login à API. Se estiver certo, devolve o nome do utilizador.
Future<String> login(String email, String password) async {
  final resposta = await http.post(
    Uri.parse('$apiUrl/login'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'email': email, 'password': password}),
  );
  final dados = jsonDecode(resposta.body);
  if (resposta.statusCode != 200) {
    throw LoginErro(dados['erro']);
  }
  utilizadorAtual = dados['id'];
  return dados['nome'];
}

/// Cria a conta na API. Se correr bem, devolve o nome do utilizador.
Future<String> registo(String nome, String email, String password) async {
  final resposta = await http.post(
    Uri.parse('$apiUrl/registo'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'nome': nome, 'email': email, 'password': password}),
  );
  final dados = jsonDecode(resposta.body);
  if (resposta.statusCode != 201) {
    throw LoginErro(dados['erro']);
  }
  utilizadorAtual = dados['id'];
  return dados['nome'];
}

/// Grava o onboarding de quem entrou.
/// [niveis]: desporto → nível (ex.: 'Padel' → 'Intermédio').
/// [horarios]: pares (dia, período) (ex.: ('Sáb', 'Manhã')).
Future<void> guardarOnboarding(
  Map<String, String> niveis,
  Set<(String, String)> horarios,
) async {
  final resposta = await http.post(
    Uri.parse('$apiUrl/onboarding'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({
      'utilizador_id': utilizadorAtual,
      'desportos': [
        for (final d in niveis.entries) {'desporto': d.key, 'nivel':
```

```
d.value},
      ],
      'disponibilidade': [
        for (final h in horarios) {'dia': h.$1, 'periodo': h.$2},
      ],
    }),
  );
  if (resposta.statusCode != 200) {
    throw LoginErro(jsonDecode(resposta.body)['erro']);
  }
}
```

Em relação à versão anterior, só mudam 3 coisas: a variável `utilizadorAtual`, a linha `utilizadorAtual = dados['id'];` no `login` e no `registo`, e a função nova `guardarOnboarding`. `(String, String)` é um *record* do Dart: um par de valores, aqui `('Sáb', 'Manhã')`.

Passo 5 — O ecrã com os 3 passos (`lib/onboarding_screen.dart`)

Substitui o `lib/onboarding_screen.dart` **inteiro** por isto. É um só ecrã: a variável `_passo` (1, 2 ou 3) escolhe o que aparece no meio, e os botões Voltar e Continuar mudam de passo sem perder as escolhas.

```
import 'package:flutter/material.dart';

import 'api.dart';
import 'cores.dart';

const desportos = [
  'Padel', 'Ténis', 'Futebol', 'BJJ',
  'Basquetebol', 'Voleibol', 'Running', 'Ginásio',
];
const niveis = ['Iniciante', 'Intermédio', 'Avançado'];
const dias = ['Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb', 'Dom'];
const periodos = ['Manhã', 'Tarde', 'Noite'];

class OnboardingScreen extends StatefulWidget {
  const OnboardingScreen({super.key});

  @override
  State<OnboardingScreen> createState() => _OnboardingScreenState();
}

class _OnboardingScreenState extends State<OnboardingScreen> {
  int _passo = 1; // 1 = desportos, 2 = nível, 3 = disponibilidade
```

```
// Passo 1 e 2: desporto escolhido → nível (começa em 'Iniciante')
final _niveis = <String, String>{};

// Passo 3: pares (dia, período), ex.: ('Sáb', 'Manhã')
final _horarios = <(String, String)>{};

bool _aGuardar = false;

void _tocarDesporto(String desporto) {
  setState(() {
    if (_niveis.containsKey(desporto)) {
      _niveis.remove(desporto);
    } else {
      _niveis[desporto] = 'Iniciante';
    }
  });
}

void _tocarHorario(String dia, String periodo) {
  setState(() {
    if (!_horarios.remove((dia, periodo))) _horarios.add((dia, periodo));
  });
}

// O botão de baixo só fica ligado quando o passo está preenchido
bool get _podeAvancar => switch (_passo) {
      1 => _niveis.isNotEmpty,
      2 => true,
      _ => _horarios.isNotEmpty && !_aGuardar,
    };

Future<void> _avancar() async {
  if (_passo < 3) {
    setState(() => _passo++);
    return;
  }
  setState(() => _aGuardar = true);
  try {
    await guardarOnboarding(_niveis, _horarios);
    if (!mounted) return;
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (_) => ProntoScreen(niveis: _niveis, horarios: _horarios),
      ),
```

```
      );
    } on LoginErro catch (erro) {
      _mostrar(erro.mensagem);
    } catch (_) {
      _mostrar('Não foi possível ligar à API. Ela está a correr?');
    } finally {
      if (mounted) setState(() => _aGuardar = false);
    }
  }

  void _mostrar(String mensagem) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(mensagem)),
    );
  }

  @override
  Widget build(BuildContext context) {
    final titulo = switch (_passo) {
      1 => 'Que desportos queres jogar?',
      2 => 'Qual é o teu nível?',
      _ => 'Quando podes jogar?',
    };
    final subtitulo = switch (_passo) {
      1 => 'Escolhe um ou mais. Podes mudar isto no teu perfil.',
      2 => 'Ajuda a encontrar jogadores parecidos contigo.',
      _ => 'Toca nos dias e períodos em que estás livre.',
    };
    final String botao;
    if (_passo < 3) {
      botao = 'Continuar';
    } else {
      botao = _aGuardar ? 'A guardar…' : 'Concluir';
    }

    return Scaffold(
      backgroundColor: fundo,
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text('Passo $_passo de 3'),
              const SizedBox(height: 8),
```

```
            LinearProgressIndicator(
              value: _passo / 3,
              color: verde,
              minHeight: 6,
            ),
            const SizedBox(height: 24),
            Text(
              titulo,
              style: const TextStyle(
                fontSize: 28,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 8),
            Text(subtitulo),
            const SizedBox(height: 24),
            Expanded(
              child: switch (_passo) {
                1 => _passoDesportos(),
                2 => _passoNivel(),
                _ => _passoHorarios(),
              },
            ),
            const SizedBox(height: 8),
            Row(
              children: [
                if (_passo > 1) ...[
                  Expanded(
                    child: OutlinedButton(
                      onPressed: () => setState(() => _passo--),
                      style: OutlinedButton.styleFrom(
                        foregroundColor: verde,
                        minimumSize: const Size.fromHeight(52),
                      ),
                      child: const Text('Voltar'),
                    ),
                  ),
                  const SizedBox(width: 12),
                ],
                Expanded(
                  child: FilledButton(
                    onPressed: _podeAvancar ? _avancar : null,
                    style: FilledButton.styleFrom(
                      backgroundColor: verde,
                      minimumSize: const Size.fromHeight(52),
                    ),
```

```
                      child: Text(botao),
                    ),
                  ),
                ],
              ),
            ],
          ),
        ),
      ),
    );
  }

  // Passo 1: grelha com os 8 desportos
  Widget _passoDesportos() {
    return GridView.count(
      crossAxisCount: 2,
      mainAxisSpacing: 12,
      crossAxisSpacing: 12,
      childAspectRatio: 2.2,
      children: [
        for (final desporto in desportos)
          CartaoOpcao(
            nome: desporto,
            escolhido: _niveis.containsKey(desporto),
            onTap: () => _tocarDesporto(desporto),
          ),
      ],
    );
  }

  // Passo 2: um nível para cada desporto escolhido
  Widget _passoNivel() {
    return ListView(
      children: [
        for (final desporto in _niveis.keys) ...[
          Text(
            desporto,
            style: const TextStyle(fontSize: 18, fontWeight:
FontWeight.bold),
          ),
          const SizedBox(height: 8),
          Wrap(
            spacing: 8,
            children: [
              for (final nivel in niveis)
                ChoiceChip(
```

```
                  label: Text(nivel),
                  selected: _niveis[desporto] == nivel,
                  selectedColor: lima,
                  onSelected: (_) => setState(() => _niveis[desporto] =
nivel),
                ),
            ],
          ),
          const SizedBox(height: 20),
        ],
      ],
    );
  }

  // Passo 3: uma linha por dia, com Manhã / Tarde / Noite
  Widget _passoHorarios() {
    return ListView(
      children: [
        for (final dia in dias)
          Padding(
            padding: const EdgeInsets.only(bottom: 8),
            child: Row(
              children: [
                SizedBox(
                  width: 44,
                  child: Text(
                    dia,
                    style: const TextStyle(fontWeight: FontWeight.bold),
                  ),
                ),
                for (final periodo in periodos)
                  Expanded(
                    child: Padding(
                      padding: const EdgeInsets.only(left: 6),
                      child: SizedBox(
                        height: 40,
                        child: CartaoOpcao(
                          nome: periodo,
                          tamanho: 14,
                          escolhido: _horarios.contains((dia, periodo)),
                          onTap: () => _tocarHorario(dia, periodo),
                        ),
                      ),
                    ),
                  ),
              ],
```

```
            ),
          ),
      ],
    );
  }
}

// Um cartão para escolher: branco quando não está escolhido, verde-lima
quando está.
// Serve para os desportos (Passo 1) e para os horários (Passo 3).
class CartaoOpcao extends StatelessWidget {
  const CartaoOpcao({
    super.key,
    required this.nome,
    required this.escolhido,
    required this.onTap,
    this.tamanho = 18,
  });

  final String nome;
  final bool escolhido;
  final VoidCallback onTap;
  final double tamanho; // tamanho da letra

  @override
  Widget build(BuildContext context) {
    return Material(
      color: escolhido ? lima : Colors.white,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
        side: BorderSide(
          color: escolhido ? Colors.black : Colors.black26,
          width: escolhido ? 2 : 1,
        ),
      ),
      child: InkWell(
        borderRadius: BorderRadius.circular(16),
        onTap: onTap,
        child: Center(
          child: Text(
            nome,
            style: TextStyle(fontSize: tamanho, fontWeight: FontWeight.bold),
          ),
        ),
      ),
    );
```

```
  }
}

// Ecrã final: mostra o que ficou guardado
class ProntoScreen extends StatelessWidget {
  const ProntoScreen({super.key, required this.niveis, required
this.horarios});

  final Map<String, String> niveis;
  final Set<(String, String)> horarios;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: fundo,
      body: SafeArea(
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            const Icon(Icons.check_circle, color: verde, size: 64),
            const SizedBox(height: 16),
            const Text(
              'Perfil guardado!',
              style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 24),
            const Text(
              'Desportos',
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
            ),
            for (final d in niveis.entries) Text('${d.key} · ${d.value}'),
            const SizedBox(height: 16),
            const Text(
              'Disponibilidade',
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
            ),
            for (final h in horarios) Text('${h.$1} · ${h.$2}'),
          ],
        ),
      ),
    );
  }
}
```

O `CartaoDesporto` da Parte 2 passou a chamar-se `CartaoOpcao` e ganhou o tamanho da letra, para servir também para Manhã / Tarde / Noite. O Login e o Criar conta não mudam: continuam a abrir `OnboardingScreen()`.

Passo 6 — Experimentar

Com o MySQL e a API ligados, no terminal da app carrega em **R** (maiúsculo) e **entra** (Login ou Criar conta): sem entrar, a app não sabe quem és.

- [ ] Passo 1: o Continuar só liga depois de escolher um desporto.
- [ ] Passo 2: aparece um grupo Iniciante / Intermédio / Avançado para cada desporto escolhido.
- [ ] Passo 3: o Concluir só liga depois de tocar num horário; o Voltar regressa ao Passo 2 sem perder o nível.
- [ ] Concluir → aparece "Perfil guardado!" com os desportos e os horários.
- [ ] No phpMyAdmin, `utilizador_desportos` e `disponibilidades` têm as linhas do teu `id`.
- [ ] Repetir o onboarding com outras escolhas → as linhas antigas desse `id` são substituídas.

O que aparece	Porquê	Como resolver
"Falta saber quem és. Entra outra vez."	A app foi reiniciada (R) e esqueceu o `utilizadorAtual`	Voltar a fazer login
"Esse utilizador não existe. Entra outra vez."	O `id` guardado já não está na tabela `utilizadores`	Fazer login com uma conta que exista
`Cannot POST /onboarding`	A API ainda tem o código antigo	Ctrl+C e `node server.js`
`Table 'whoplay_login.utilizador_desportos' doesn't exist`	Faltou o Passo 1	Correr o SQL do Passo 1 com `whoplay_login` selecionada

Simplificação deste guia: a app envia o `id` de quem entrou, e a API confia nele. Numa app a sério, o login devolve um *token* (JWT) e a API descobre quem é o utilizador a partir dele, como faz o backend completo do projeto (`whoplay/backend`).

Parte 6 — Perfil

O ecrã Perfil mostra os dados de quem entrou, lidos da base de dados: um cartão verde com a inicial, o nome e o email, e as secções **Desportos e nível** e **Disponibilidade**. Tem os botões **Editar perfil** (abre o Onboarding outra vez) e **Terminar sessão** (volta ao Login).

A navegação passa a ser esta:

De	Para
Login → Entrar	Perfil
Criar conta	Onboarding → Perfil
Perfil → Editar perfil	Onboarding → Perfil (com os dados novos)
Perfil → Terminar sessão	Login

Não é preciso mexer na base de dados: as tabelas da Parte 5 já têm tudo.

Passo 1 — A rota `GET /perfil/:id` (`server.js`)

Em `~/Documents/whoplay_login_api/server.js`, cola isto **antes** de `app.listen(3000, ...)`:

```
// GET /perfil/1  →  nome, email, desportos e disponibilidade do utilizador 1
app.get('/perfil/:id', async (req, res) => {
  const id = req.params.id;

  const [utilizadores] = await db.query(
    'SELECT id, nome, email FROM utilizadores WHERE id = ?',
    [id],
  );
  if (utilizadores.length === 0) {
    return res.status(404).json({ erro: 'Esse utilizador não existe. Entra
outra vez.' });
  }

  const [desportos] = await db.query(
    'SELECT desporto, nivel FROM utilizador_desportos WHERE utilizador_id = ?
ORDER BY desporto',
    [id],
  );

  // Ordena pelos dias da semana e pelos períodos do dia (e não por ordem
alfabética)
  const [disponibilidade] = await db.query(
```

```
    `SELECT dia, periodo FROM disponibilidades WHERE utilizador_id = ?
     ORDER BY FIELD(dia, 'Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb', 'Dom'),
              FIELD(periodo, 'Manhã', 'Tarde', 'Noite')`,
    [id],
  );

  res.json({ ...utilizadores[0], desportos, disponibilidade });
});
```

É a primeira rota `GET`: só lê, não recebe corpo, e o `id` vem no próprio endereço (`/perfil/1`), em `req.params.id`. O `FIELD(...)` ordena pela ordem da lista: sem ele, o MySQL punha "Dom" antes de "Seg". O `...utilizadores[0]` copia `id`, `nome` e `email` para a resposta.

No terminal da API, **Ctrl+C** e volta a ligar:

```
node server.js
```

Passo 2 — Testar a rota

Um `GET` testa-se só com o endereço. Abre no Chrome `http://localhost:3000/perfil/1`, ou no Git Bash:

```
curl http://localhost:3000/perfil/1
```

O que deves ver: algo como `{"id":1,"nome":"Sebastião","email":"sebastiao@email.com","desportos":[...],"disponibilidade":[...]}`. Com um `id` que não existe (por exemplo `/perfil/99`): `{"erro":"Esse utilizador não existe. Entra outra vez."}`.

Passo 3 — Ler o perfil na app (`lib/api.dart`)

Cola isto no **fim** de `lib/api.dart`:

```
/// Os dados que aparecem no ecrã Perfil.
class Perfil {
  Perfil({
    required this.nome,
    required this.email,
    required this.desportos,
    required this.horarios,
  });

  final String nome;
  final String email;
```

```
  final Map<String, String> desportos; // desporto → nível
  final List<(String, String)> horarios; // (dia, período)
}

/// Vai buscar à API o perfil de quem entrou.
Future<Perfil> buscarPerfil() async {
  if (utilizadorAtual == null) {
    throw LoginErro('Falta saber quem és. Entra outra vez.');
  }
  final resposta = await
http.get(Uri.parse('$apiUrl/perfil/$utilizadorAtual'));
  final dados = jsonDecode(resposta.body);
  if (resposta.statusCode != 200) {
    throw LoginErro(dados['erro']);
  }
  return Perfil(
    nome: dados['nome'],
    email: dados['email'],
    desportos: {
      for (final d in dados['desportos']) d['desporto']: d['nivel'],
    },
    horarios: [
      for (final h in dados['disponibilidade']) (h['dia'], h['periodo']),
    ],
  );
}

/// Terminar sessão: a app esquece quem entrou.
void terminarSessao() {
  utilizadorAtual = null;
}
```

A classe `Perfil` junta numa só variável tudo o que o ecrã precisa. A `buscarPerfil` converte o JSON da API nessa classe: a lista de desportos vira um mapa (`Padel → Intermédio`) e a disponibilidade vira uma lista de pares (`('Sáb', 'Manhã')`).

Passo 4 — O ecrã (`lib/perfil_screen.dart`)

Cria o ficheiro `lib/perfil_screen.dart`:

```
import 'package:flutter/material.dart';

import 'api.dart';
import 'cores.dart';
import 'login_screen.dart';
import 'onboarding_screen.dart';
```

```
class PerfilScreen extends StatefulWidget {
  const PerfilScreen({super.key});

  @override
  State<PerfilScreen> createState() => _PerfilScreenState();
}

class _PerfilScreenState extends State<PerfilScreen> {
  // O pedido à API começa logo que o ecrã abre
  late Future<Perfil> _perfil = buscarPerfil();

  void _editar() {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => const OnboardingScreen()),
    );
  }

  void _sair() {
    terminarSessao();
    // Volta ao Login e apaga o histórico (o "voltar" já não regressa ao
Perfil)
    Navigator.pushAndRemoveUntil(
      context,
      MaterialPageRoute(builder: (_) => const LoginScreen()),
      (_) => false,
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: fundo,
      body: SafeArea(
        // FutureBuilder: mostra uma coisa enquanto espera e outra quando
chega
        child: FutureBuilder<Perfil>(
          future: _perfil,
          builder: (context, pedido) {
            if (pedido.connectionState != ConnectionState.done) {
              return const Center(child: CircularProgressIndicator(color:
verde));
            }
            if (pedido.hasError) {
              final erro = pedido.error;
```

```
              return ErroPerfil(
                mensagem: erro is LoginErro
                    ? erro.mensagem
                    : 'Não foi possível ligar à API. Ela está a correr?',
                onTentar: () => setState(() => _perfil = buscarPerfil()),
                onSair: _sair,
              );
            }
            return CorpoPerfil(
              perfil: pedido.data!,
              onEditar: _editar,
              onSair: _sair,
            );
          },
        ),
      ),
    );
  }
}

// O conteúdo do Perfil, depois de os dados chegarem
class CorpoPerfil extends StatelessWidget {
  const CorpoPerfil({
    super.key,
    required this.perfil,
    required this.onEditar,
    required this.onSair,
  });

  final Perfil perfil;
  final VoidCallback onEditar;
  final VoidCallback onSair;

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.all(24),
      children: [
        const Text(
          'Perfil',
          style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
        ),
        const SizedBox(height: 16),

        // Cartão verde com a inicial, o nome e o email
        Container(
```

```
          padding: const EdgeInsets.all(18),
          decoration: BoxDecoration(
            color: floresta,
            borderRadius: BorderRadius.circular(20),
          ),
          child: Row(
            children: [
              CircleAvatar(
                radius: 32,
                backgroundColor: lima,
                child: Text(
                  perfil.nome.substring(0, 1),
                  style: const TextStyle(
                    fontSize: 28,
                    fontWeight: FontWeight.bold,
                    color: Colors.black,
                  ),
                ),
              ),
              const SizedBox(width: 16),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      perfil.nome,
                      style: const TextStyle(
                        fontSize: 22,
                        fontWeight: FontWeight.bold,
                        color: Colors.white,
                      ),
                    ),
                    const SizedBox(height: 4),
                    Text(
                      perfil.email,
                      style: const TextStyle(color: Colors.white),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
        const SizedBox(height: 24),

        const Titulo('Desportos e nível'),
```

```
        if (perfil.desportos.isEmpty)
          const Text('Ainda não escolheste desportos.')
        else
          Cartao(
            children: [
              for (final d in perfil.desportos.entries)
                Linha(
                  esquerda: d.key,
                  direita: Etiqueta(d.value, destaque: d.value !=
'Iniciante'),
                ),
            ],
          ),
        const SizedBox(height: 24),

        const Titulo('Disponibilidade'),
        if (perfil.horarios.isEmpty)
          const Text('Ainda não indicaste horários.')
        else
          Cartao(
            children: [
              // Uma linha por dia, com os períodos desse dia: "Sáb  Manhã,
Tarde"
              for (final dia in dias)
                if (perfil.horarios.any((h) => h.$1 == dia))
                  Linha(
                    esquerda: dia,
                    direita: Text([
                      for (final h in perfil.horarios)
                        if (h.$1 == dia) h.$2,
                    ].join(', ')),
                  ),
            ],
          ),
        const SizedBox(height: 32),

        FilledButton(
          onPressed: onEditar,
          style: FilledButton.styleFrom(
            backgroundColor: verde,
            minimumSize: const Size.fromHeight(52),
          ),
          child: const Text('Editar perfil'),
        ),
        const SizedBox(height: 12),
        OutlinedButton(
```

```
          onPressed: onSair,
          style: OutlinedButton.styleFrom(
            foregroundColor: verde,
            minimumSize: const Size.fromHeight(52),
          ),
          child: const Text('Terminar sessão'),
        ),
      ],
    );
  }
}

// Mostrado quando o pedido à API falha
class ErroPerfil extends StatelessWidget {
  const ErroPerfil({
    super.key,
    required this.mensagem,
    required this.onTentar,
    required this.onSair,
  });

  final String mensagem;
  final VoidCallback onTentar;
  final VoidCallback onSair;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(24),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Text(mensagem, textAlign: TextAlign.center),
          const SizedBox(height: 16),
          FilledButton(
            onPressed: onTentar,
            style: FilledButton.styleFrom(backgroundColor: verde),
            child: const Text('Tentar outra vez'),
          ),
          TextButton(onPressed: onSair, child: const Text('Voltar ao
Login')),
        ],
      ),
    );
  }
```

```
}

// Peças pequenas usadas acima

class Titulo extends StatelessWidget {
  const Titulo(this.texto, {super.key});

  final String texto;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 8),
      child: Text(
        texto.toUpperCase(),
        style: const TextStyle(
          fontSize: 13,
          fontWeight: FontWeight.bold,
          color: Colors.black54,
        ),
      ),
    );
  }
}

class Cartao extends StatelessWidget {
  const Cartao({super.key, required this.children});

  final List<Widget> children;

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: Colors.black12),
      ),
      child: Column(children: children),
    );
  }
}

class Linha extends StatelessWidget {
  const Linha({super.key, required this.esquerda, required this.direita});
```

```
  final String esquerda;
  final Widget direita;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Text(
            esquerda,
            style: const TextStyle(fontSize: 16, fontWeight:
FontWeight.bold),
          ),
          direita,
        ],
      ),
    );
  }
}

class Etiqueta extends StatelessWidget {
  const Etiqueta(this.texto, {super.key, this.destaque = false});

  final String texto;
  final bool destaque;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 5),
      decoration: BoxDecoration(
        color: destaque ? lima : Colors.black12,
        borderRadius: BorderRadius.circular(14),
      ),
      child: Text(texto, style: const TextStyle(fontWeight:
FontWeight.bold)),
    );
  }
}
```

O `FutureBuilder` mostra uma roda a girar enquanto a API responde, a mensagem de erro (com "Tentar outra vez") se falhar, e o `CorpoPerfil` quando os dados chegam. A lista `dias` vem do `onboarding_screen.dart` e serve para mostrar a disponibilidade de segunda a domingo.

Passo 5 — Ligar o Login e o Onboarding ao Perfil

a) **`lib/login_screen.dart`** — o Entrar passa a abrir o Perfil. Troca a linha `import 'onboarding_screen.dart';` por:

```
import 'perfil_screen.dart';
```

E, dentro da função `_entrar`, troca `const OnboardingScreen()` por `const PerfilScreen()`:

```
        MaterialPageRoute(builder: (_) => const PerfilScreen()),
```

b) **`lib/onboarding_screen.dart`** — o Concluir passa a abrir o Perfil. Junta este import no topo, por baixo de `import 'cores.dart';`:

```
import 'perfil_screen.dart';
```

Dentro da função `_avancar`, troca este bloco:

```
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(
          builder: (_) => ProntoScreen(niveis: _niveis, horarios: _horarios),
        ),
      );
```

por este:

```
      _mostrar('Perfil guardado!');
      Navigator.pushAndRemoveUntil(
        context,
        MaterialPageRoute(builder: (_) => const PerfilScreen()),
        (_) => false,
      );
```

Por fim, apaga a classe `ProntoScreen` inteira (do comentário `// Ecrã final: mostra o que ficou guardado` até ao fim do ficheiro): o Perfil faz agora esse papel. O `register_screen.dart` não muda: o Criar conta continua a abrir o Onboarding.

Passo 6 — Experimentar

Com o MySQL e a API ligados, no terminal da app carrega em **R** (maiúsculo).

- [ ] Login com o Sebastião → abre o Perfil com o cartão verde, o nome e o email.
- [ ] Os desportos aparecem com o nível; os que não são Iniciante ficam com a etiqueta verde-lima.

- [ ] A disponibilidade aparece de Seg a Dom, com os períodos juntos (ex.: "Sáb — Manhã, Tarde").
- [ ] Editar perfil → Onboarding → Concluir → o Perfil volta a abrir com as escolhas novas.
- [ ] Terminar sessão → volta ao Login; o "voltar" do navegador já não regressa ao Perfil.
- [ ] Criar conta nova → Onboarding → Concluir → Perfil da conta nova.
- [ ] Desliga a API e abre o Perfil → aparece "Não foi possível ligar à API" com "Tentar outra vez".

O que aparece	Porquê	Como resolver
`Interm�dio` ou `S�b` no Perfil	Linhas gravadas pelo `curl` com acentos (Parte 5)	Editar perfil e concluir o Onboarding outra vez
"Falta saber quem és. Entra outra vez."	A app foi reiniciada (R) e esqueceu quem entrou	Voltar ao Login e entrar
`Cannot GET /perfil/1`	A API ainda tem o código antigo	Ctrl+C e `node server.js`
`The name 'ProntoScreen' isn't a class`	Apagaste a classe mas ficou o bloco antigo no `_avancar`	Fazer a troca do bloco do Passo 5 b)
`Undefined name 'dias'` no `perfil_screen.dart`	Falta o `import 'onboarding_screen.dart';`	Copiar o ficheiro inteiro outra vez do guia online

Parte 7 — Visual dos mockups (Onboarding e Perfil)

Nesta parte o Onboarding e o Perfil ficam com o aspeto dos mockups 02 e 03. **Só muda o visual**: a API, a base de dados e o `api.dart` ficam iguais, e o que cada botão faz também.

O que muda	Como no mockup
Letras	**Bricolage Grotesque** nos títulos e **DM Sans** no resto (pacote `google_fonts`)
Onboarding	Barra "Passo 1 de 3" em 3 traços, cartões com a frase de cada desporto e ✓, frase "2 desportos escolhidos · a seguir: …" por cima do botão

O que muda	Como no mockup
Perfil	"Perfil" com a pílula "Editar", cartão verde-escuro com avatar lima, rótulos em maiúsculas, cartões com linhas, dias por extenso
Botões	56 de altura e cantos de 14; cinzentos quando estão desligados
Mensagens de baixo	Fundo escuro e cantos arredondados

Ficheiro	O que fazer
`lib/cores.dart`	Substituir (mais 6 cores)
`lib/estilo.dart`	Criar (letras e peças partilhadas)
`lib/main.dart`	Substituir (tema com a letra DM Sans)
`lib/onboarding_screen.dart`	Substituir
`lib/perfil_screen.dart`	Substituir

Passo 1 — Instalar as letras

Na pasta `whoplay_login`:

```
flutter pub add google_fonts:^6.3.3
```

Usa esta versão: a mais recente (9.0) dá o erro `The argument type 'TextTheme' can't be assigned to the parameter type 'TextTheme?'` com o Flutter 3.47. As letras são descarregadas da internet na primeira vez que a app abre.

Passo 2 — As cores (`lib/cores.dart`)

Substitui o ficheiro inteiro. As 4 cores de antes ficam iguais; juntam-se as cores de texto, contornos e etiquetas dos mockups:

```
import 'package:flutter/material.dart';

// Cores do WhoPlay (as mesmas dos mockups).
const verde = Color(0xFF1F7A4D);
const floresta = Color(0xFF123D29);
const lima = Color(0xFFC9F25C);
const fundo = Color(0xFFF3F5F0);

const tinta = Color(0xFF10201A); // texto principal
const cinza = Color(0xFF55635C); // texto secundário
const linha = Color(0xFFDDE3DA); // contorno dos cartões
const contorno = Color(0xFFB9C4BA); // contorno das opções por escolher
const trilho = Color(0xFFCDD6CB); // parte vazia da barra de progresso
const chip = Color(0xFFE6ECE3); // fundo das etiquetas cinzentas
```

Passo 3 — Letras e peças partilhadas (`lib/estilo.dart`)

Cria o ficheiro `lib/estilo.dart`. Tem as 2 letras (`titulo(...)` e `texto(...)`) e as peças que o Onboarding e o Perfil usam: `Botao` , `Rotulo` , `CartaoLista` , `Etiqueta` e `Opcao` .

```
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

import 'cores.dart';

// Letra dos títulos (Bricolage Grotesque), como nos mockups
TextStyle titulo(double tamanho, {Color cor = tinta}) {
  return GoogleFonts.getFont(
    'Bricolage Grotesque',
    fontSize: tamanho,
    fontWeight: FontWeight.w700,
    color: cor,
    letterSpacing: -0.5,
    height: 1.15,
  );
}

// Letra do resto do texto (DM Sans)
TextStyle texto(
  double tamanho, {
  FontWeight peso = FontWeight.w400,
  Color cor = tinta,
}) {
  return GoogleFonts.dmSans(fontSize: tamanho, fontWeight: peso, color: cor);
```

```
}

// Botão principal (verde) ou contornado: 56 de altura, cantos de 14
class Botao extends StatelessWidget {
  const Botao({
    super.key,
    required this.texto,
    required this.onPressed,
    this.contornado = false,
  });

  final String texto;
  final VoidCallback? onPressed;
  final bool contornado;

  @override
  Widget build(BuildContext context) {
    final forma = RoundedRectangleBorder(borderRadius:
BorderRadius.circular(14));
    final letra = GoogleFonts.dmSans(fontSize: 17, fontWeight:
FontWeight.w700);
    const tamanho = Size.fromHeight(56);
    if (contornado) {
      return OutlinedButton(
        onPressed: onPressed,
        style: OutlinedButton.styleFrom(
          foregroundColor: tinta,
          side: const BorderSide(color: tinta, width: 1.5),
          shape: forma,
          textStyle: letra,
          minimumSize: tamanho,
        ),
        child: Text(texto),
      );
    }
    return FilledButton(
      onPressed: onPressed,
      style: FilledButton.styleFrom(
        backgroundColor: verde,
        disabledBackgroundColor: trilho,
        shape: forma,
        textStyle: letra,
        minimumSize: tamanho,
      ),
      child: Text(texto),
    );
```

```
  }
}

// Título pequeno de secção, em maiúsculas: "DESPORTOS E NÍVEL"
class Rotulo extends StatelessWidget {
  const Rotulo(this.texto, {super.key});

  final String texto;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 10),
      child: Text(
        texto.toUpperCase(),
        style: GoogleFonts.dmSans(
          fontSize: 13,
          fontWeight: FontWeight.w700,
          color: cinza,
          letterSpacing: 0.8,
        ),
      ),
    );
  }
}

// Cartão branco com as linhas separadas por um traço fino
class CartaoLista extends StatelessWidget {
  const CartaoLista({super.key, required this.children});

  final List<Widget> children;

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: linha),
      ),
      child: Column(
        children: [
          for (var i = 0; i < children.length; i++) ...[
            if (i > 0) const Divider(height: 1, thickness: 1, color: linha),
            children[i],
          ],
```

```
        ],
      ),
    );
  }
}

// Etiqueta arredondada: verde-lima (destaque) ou cinzenta
class Etiqueta extends StatelessWidget {
  const Etiqueta(this.texto, {super.key, this.destaque = false});

  final String texto;
  final bool destaque;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 5),
      decoration: BoxDecoration(
        color: destaque ? lima : chip,
        borderRadius: BorderRadius.circular(14),
      ),
      child: Text(
        texto,
        style: GoogleFonts.dmSans(fontSize: 14, fontWeight: FontWeight.w700),
      ),
    );
  }
}

// Opção para tocar: branca por escolher, verde-lima com contorno escuro
quando escolhida
class Opcao extends StatelessWidget {
  const Opcao({
    super.key,
    required this.escolhida,
    required this.onTap,
    required this.child,
    this.altura = 48,
    this.raio = 12,
  });

  final bool escolhida;
  final VoidCallback onTap;
  final Widget child;
  final double altura;
  final double raio;
```

```
  @override
  Widget build(BuildContext context) {
    return Material(
      color: escolhida ? lima : Colors.white,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(raio),
        side: escolhida
            ? const BorderSide(color: tinta, width: 2)
            : const BorderSide(color: contorno),
      ),
      clipBehavior: Clip.antiAlias,
      child: InkWell(
        onTap: onTap,
        child: SizedBox(height: altura, child: child),
      ),
    );
  }
}
```

Passo 4 — A letra em toda a app (`lib/main.dart`)

Substitui o ficheiro inteiro. O `theme` põe a DM Sans em todos os ecrãs (o Login e o Criar conta também mudam de letra) e dá às mensagens de baixo o estilo dos mockups:

```
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

import 'cores.dart';
import 'login_screen.dart';

void main() {
  runApp(const WhoPlayApp());
}

class WhoPlayApp extends StatelessWidget {
  const WhoPlayApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'WhoPlay',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: verde,
        scaffoldBackgroundColor: fundo,
```

```
        // DM Sans em toda a app, como nos mockups
        textTheme: GoogleFonts.dmSansTextTheme(),
        // Mensagens de baixo: fundo escuro e cantos arredondados
        snackBarTheme: SnackBarThemeData(
          backgroundColor: tinta,
          behavior: SnackBarBehavior.floating,
          contentTextStyle: GoogleFonts.dmSans(
            fontSize: 15,
            fontWeight: FontWeight.w600,
            color: Colors.white,
          ),
          shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(12)),
        ),
      ),
      home: const LoginScreen(),
    );
  }
}
```

Passo 5 — O Onboarding como o mockup 02 (`lib/onboarding_screen.dart`)

Substitui o ficheiro inteiro. A lógica dos 3 passos é a da Parte 5; muda o aspeto, os desportos passam a ter a frase pequena do mockup ("2 ou 4 jogadores"), e o Passo 1 ganha um "Voltar" quando se chega pelo "Editar" do Perfil.

```
import 'package:flutter/material.dart';

import 'api.dart';
import 'cores.dart';
import 'estilo.dart';
import 'perfil_screen.dart';

// Os 8 desportos e a frase pequena de cada um (como no mockup)
const desportos = {
  'Padel': '2 ou 4 jogadores',
  'Ténis': '2 ou 4 jogadores',
  'Futebol': '5, 7 ou 11 por equipa',
  'BJJ': 'Parceiro de treino',
  'Basquetebol': '3x3 ou 5x5',
  'Voleibol': 'Praia ou pavilhão',
  'Running': 'Companhia de corrida',
  'Ginásio': 'Parceiro de treino',
};
const niveis = ['Iniciante', 'Intermédio', 'Avançado'];
const dias = ['Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb', 'Dom'];
```

```
const periodos = ['Manhã', 'Tarde', 'Noite'];

class OnboardingScreen extends StatefulWidget {
  const OnboardingScreen({super.key});

  @override
  State<OnboardingScreen> createState() => _OnboardingScreenState();
}

class _OnboardingScreenState extends State<OnboardingScreen> {
  int _passo = 1; // 1 = desportos, 2 = nível, 3 = disponibilidade

  // Passo 1 e 2: desporto escolhido → nível (começa em 'Iniciante')
  final _niveis = <String, String>{};

  // Passo 3: pares (dia, período), ex.: ('Sáb', 'Manhã')
  final _horarios = <(String, String)>{};

  bool _aGuardar = false;

  void _tocarDesporto(String desporto) {
    setState(() {
      if (_niveis.containsKey(desporto)) {
        _niveis.remove(desporto);
      } else {
        _niveis[desporto] = 'Iniciante';
      }
    });
  }

  void _tocarHorario(String dia, String periodo) {
    setState(() {
      if (!_horarios.remove((dia, periodo))) _horarios.add((dia, periodo));
    });
  }

  // O botão de baixo só fica ligado quando o passo está preenchido
  bool get _podeAvancar => switch (_passo) {
        1 => _niveis.isNotEmpty,
        2 => true,
        _ => _horarios.isNotEmpty && !_aGuardar,
      };

  Future<void> _avancar() async {
    if (_passo < 3) {
      setState(() => _passo++);
```

```
      return;
    }
    setState(() => _aGuardar = true);
    try {
      await guardarOnboarding(_niveis, _horarios);
      if (!mounted) return;
      _mostrar('Perfil guardado!');
      Navigator.pushAndRemoveUntil(
        context,
        MaterialPageRoute(builder: (_) => const PerfilScreen()),
        (_) => false,
      );
    } on LoginErro catch (erro) {
      _mostrar(erro.mensagem);
    } catch (_) {
      _mostrar('Não foi possível ligar à API. Ela está a correr?');
    } finally {
      if (mounted) setState(() => _aGuardar = false);
    }
  }

  void _mostrar(String mensagem) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(mensagem)),
    );
  }

  @override
  Widget build(BuildContext context) {
    final tituloDoPasso = switch (_passo) {
      1 => 'Que desportos queres jogar?',
      2 => 'Qual é o teu nível?',
      _ => 'Quando podes jogar?',
    };
    final subtitulo = switch (_passo) {
      1 => 'Escolhe um ou mais. Podes mudar isto no teu perfil.',
      2 => 'Ajuda a encontrar jogadores do teu nível.',
      _ => 'Toca nos dias e períodos em que estás livre.',
    };

    // A frase por cima do botão, como no mockup
    final n = _niveis.length;
    final escolhidos = n == 1 ? '1 desporto escolhido' : '$n desportos
escolhidos';
    final resumo = switch (_passo) {
```

```
      1 => '$escolhidos · a seguir: nível e disponibilidade',
      2 => 'A seguir: disponibilidade',
      _ => _horarios.length == 1
          ? '1 horário escolhido'
          : '${_horarios.length} horários escolhidos',
    };

    // No Passo 1 só há "Voltar" quando se veio do Perfil (Editar)
    final podeVoltar = _passo > 1 || Navigator.canPop(context);

    final String botao;
    if (_passo < 3) {
      botao = 'Continuar';
    } else {
      botao = _aGuardar ? 'A guardar…' : 'Concluir';
    }

    return Scaffold(
      backgroundColor: fundo,
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.fromLTRB(24, 12, 24, 24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              BarraPassos(passo: _passo, total: 3),
              const SizedBox(height: 28),
              Text(tituloDoPasso, style: titulo(30)),
              const SizedBox(height: 8),
              Text(subtitulo, style: texto(16, cor: cinza)),
              const SizedBox(height: 24),
              Expanded(
                child: switch (_passo) {
                  1 => _passoDesportos(),
                  2 => _passoNivel(),
                  _ => _passoHorarios(),
                },
              ),
              const SizedBox(height: 16),
              Text(
                resumo,
                textAlign: TextAlign.center,
                style: texto(14, cor: cinza),
              ),
              const SizedBox(height: 10),
              Row(
```

```
                children: [
                  if (podeVoltar) ...[
                    Expanded(
                      child: Botao(
                        texto: 'Voltar',
                        contornado: true,
                        onPressed: _passo > 1
                            ? () => setState(() => _passo--)
                            : () => Navigator.pop(context),
                      ),
                    ),
                    const SizedBox(width: 12),
                  ],
                  Expanded(
                    flex: 2,
                    child: Botao(
                      texto: botao,
                      onPressed: _podeAvancar ? _avancar : null,
                    ),
                  ),
                ],
              ),
            ],
          ),
        ),
      ),
    );
  }

  // Passo 1: grelha com os 8 desportos
  Widget _passoDesportos() {
    return GridView(
      // 2 colunas e cartões sempre com 88 de altura
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: 2,
        mainAxisSpacing: 12,
        crossAxisSpacing: 12,
        mainAxisExtent: 88,
      ),
      children: [
        for (final d in desportos.entries)
          CartaoDesporto(
            nome: d.key,
            descricao: d.value,
            escolhido: _niveis.containsKey(d.key),
            onTap: () => _tocarDesporto(d.key),
```

```
        ),
    ],
  );
}

// Passo 2: para cada desporto escolhido, 3 opções de nível lado a lado
Widget _passoNivel() {
  return ListView(
    children: [
      for (final desporto in _niveis.keys) ...[
        Rotulo(desporto),
        Row(
          children: [
            for (final nivel in niveis) ...[
              if (nivel != niveis.first) const SizedBox(width: 8),
              Expanded(
                child: Opcao(
                  escolhida: _niveis[desporto] == nivel,
                  onTap: () => setState(() => _niveis[desporto] = nivel),
                  child: Center(
                    child: Text(
                      nivel,
                      style: texto(14, peso: FontWeight.w700),
                    ),
                  ),
                ),
              ),
            ],
          ],
        ),
        const SizedBox(height: 20),
      ],
    ],
  );
}

// Passo 3: uma linha por dia, com Manhã / Tarde / Noite
Widget _passoHorarios() {
  return ListView(
    children: [
      for (final dia in dias)
        Padding(
          padding: const EdgeInsets.only(bottom: 8),
          child: Row(
            children: [
              SizedBox(
```

```
                  width: 44,
                  child: Text(dia, style: texto(15, peso: FontWeight.w700)),
                ),
                for (final periodo in periodos)
                  Expanded(
                    child: Padding(
                      padding: const EdgeInsets.only(left: 8),
                      child: Opcao(
                        altura: 44,
                        escolhida: _horarios.contains((dia, periodo)),
                        onTap: () => _tocarHorario(dia, periodo),
                        child: Center(
                          child: Text(
                            periodo,
                            style: texto(14, peso: FontWeight.w600),
                          ),
                        ),
                      ),
                    ),
                  ),
              ],
            ),
          ),
      ],
    );
  }
}

// A barra "Passo 1 de 3": 3 traços, os já feitos a verde
class BarraPassos extends StatelessWidget {
  const BarraPassos({super.key, required this.passo, required this.total});

  final int passo;
  final int total;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        for (var i = 0; i < total; i++) ...[
          if (i > 0) const SizedBox(width: 6),
          Expanded(
            child: Container(
              height: 6,
              decoration: BoxDecoration(
                color: i < passo ? verde : trilho,
```

```
                borderRadius: BorderRadius.circular(3),
              ),
            ),
          ),
        ],
        const SizedBox(width: 12),
        Text(
          'Passo $passo de $total',
          style: texto(13, peso: FontWeight.w600, cor: cinza),
        ),
      ],
    );
  }
}

// Cartão de um desporto: nome, frase pequena e ✓ quando está escolhido
class CartaoDesporto extends StatelessWidget {
  const CartaoDesporto({
    super.key,
    required this.nome,
    required this.descricao,
    required this.escolhido,
    required this.onTap,
  });

  final String nome;
  final String descricao;
  final bool escolhido;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Opcao(
      altura: 88,
      raio: 16,
      escolhida: escolhido,
      onTap: onTap,
      child: Padding(
        padding: const EdgeInsets.all(14),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Row(
              children: [
                Expanded(
```

```
                  child: Text(
                    nome,
                    overflow: TextOverflow.ellipsis,
                    style: titulo(19),
                  ),
                ),
                if (escolhido) const Icon(Icons.check, size: 20, color:
tinta),
              ],
            ),
            Text(
              descricao,
              maxLines: 2,
              overflow: TextOverflow.ellipsis,
              style: texto(
                13,
                peso: FontWeight.w500,
                cor: escolhido ? tinta : cinza,
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

`desportos` passou de lista a **mapa** (nome → frase). `Navigator.canPop(context)` diz se há um ecrã por baixo: é verdade quando se veio do Perfil e falso quando se veio do Criar conta.

Passo 6 — O Perfil como o mockup 03 (`lib/perfil_screen.dart`)

Substitui o ficheiro inteiro. As peças `Titulo` , `Cartao` , `Linha` e `Etiqueta` da Parte 6 saem daqui: as novas estão no `estilo.dart` .

```
import 'package:flutter/material.dart';

import 'api.dart';
import 'cores.dart';
import 'estilo.dart';
import 'login_screen.dart';
import 'onboarding_screen.dart';

// Nome do dia por extenso, para a secção Disponibilidade
const diasPorExtenso = {
```

```
  'Seg': 'Segunda',
  'Ter': 'Terça',
  'Qua': 'Quarta',
  'Qui': 'Quinta',
  'Sex': 'Sexta',
  'Sáb': 'Sábado',
  'Dom': 'Domingo',
};

class PerfilScreen extends StatefulWidget {
  const PerfilScreen({super.key});

  @override
  State<PerfilScreen> createState() => _PerfilScreenState();
}

class _PerfilScreenState extends State<PerfilScreen> {
  // O pedido à API começa logo que o ecrã abre
  late Future<Perfil> _perfil = buscarPerfil();

  void _editar() {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => const OnboardingScreen()),
    );
  }

  void _sair() {
    terminarSessao();
    // Volta ao Login e apaga o histórico (o "voltar" já não regressa ao
Perfil)
    Navigator.pushAndRemoveUntil(
      context,
      MaterialPageRoute(builder: (_) => const LoginScreen()),
      (_) => false,
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: fundo,
      body: SafeArea(
        // FutureBuilder: mostra uma coisa enquanto espera e outra quando
chega
        child: FutureBuilder<Perfil>(
```

```
          future: _perfil,
          builder: (context, pedido) {
            if (pedido.connectionState != ConnectionState.done) {
              return const Center(child: CircularProgressIndicator(color:
verde));
            }
            if (pedido.hasError) {
              final erro = pedido.error;
              return ErroPerfil(
                mensagem: erro is LoginErro
                    ? erro.mensagem
                    : 'Não foi possível ligar à API. Ela está a correr?',
                onTentar: () => setState(() => _perfil = buscarPerfil()),
                onSair: _sair,
              );
            }
            return CorpoPerfil(
              perfil: pedido.data!,
              onEditar: _editar,
              onSair: _sair,
            );
          },
        ),
      ),
    );
  }
}

// O conteúdo do Perfil, depois de os dados chegarem
class CorpoPerfil extends StatelessWidget {
  const CorpoPerfil({
    super.key,
    required this.perfil,
    required this.onEditar,
    required this.onSair,
  });

  final Perfil perfil;
  final VoidCallback onEditar;
  final VoidCallback onSair;

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.fromLTRB(20, 16, 20, 20),
      children: [
```

```
        // "Perfil" à esquerda e o botão "Editar" em pílula à direita
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Text('Perfil', style: titulo(28)),
            OutlinedButton(
              onPressed: onEditar,
              style: OutlinedButton.styleFrom(
                foregroundColor: tinta,
                side: const BorderSide(color: tinta, width: 1.5),
                shape: const StadiumBorder(),
                minimumSize: const Size(0, 44),
                padding: const EdgeInsets.symmetric(horizontal: 18),
                textStyle: texto(15, peso: FontWeight.w700),
              ),
              child: const Text('Editar'),
            ),
          ],
        ),
        const SizedBox(height: 18),

        // Cartão verde-escuro com a inicial, o nome e o email
        Container(
          padding: const EdgeInsets.all(18),
          decoration: BoxDecoration(
            color: floresta,
            borderRadius: BorderRadius.circular(20),
          ),
          child: Row(
            children: [
              Container(
                width: 64,
                height: 64,
                alignment: Alignment.center,
                decoration: const BoxDecoration(color: lima, shape:
BoxShape.circle),
                child: Text(perfil.nome.substring(0, 1), style: titulo(28)),
              ),
              const SizedBox(width: 16),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(perfil.nome, style: titulo(22, cor: Colors.white)),
                    const SizedBox(height: 4),
                    Row(
```

```
                      children: [
                        const Icon(Icons.mail_outline, size: 16, color:
Colors.white),
                        const SizedBox(width: 6),
                        Flexible(
                          child: Text(
                            perfil.email,
                            overflow: TextOverflow.ellipsis,
                            style: texto(14, cor: Colors.white),
                          ),
                        ),
                      ],
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
        const SizedBox(height: 18),

        const Rotulo('Desportos e nível'),
        if (perfil.desportos.isEmpty)
          Text('Ainda não escolheste desportos.', style: texto(15, cor:
cinza))
        else
          CartaoLista(
            children: [
              for (final d in perfil.desportos.entries)
                Padding(
                  padding: const EdgeInsets.all(16),
                  child: Row(
                    mainAxisAlignment: MainAxisAlignment.spaceBetween,
                    children: [
                      Text(d.key, style: texto(17, peso: FontWeight.w700)),
                      Etiqueta(d.value, destaque: d.value != 'Iniciante'),
                    ],
                  ),
                ),
            ],
          ),
        const SizedBox(height: 18),

        const Rotulo('Disponibilidade'),
        if (perfil.horarios.isEmpty)
          Text('Ainda não indicaste horários.', style: texto(15, cor: cinza))
```

```
        else
          CartaoLista(
            children: [
              // Uma linha por dia, com os períodos desse dia: "Sábado
Manhã, Tarde"
              for (final dia in dias)
                if (perfil.horarios.any((h) => h.$1 == dia))
                  Padding(
                    padding: const EdgeInsets.symmetric(horizontal: 16,
vertical: 14),
                    child: Row(
                      mainAxisAlignment: MainAxisAlignment.spaceBetween,
                      children: [
                        Text(
                          diasPorExtenso[dia]!,
                          style: texto(16, peso: FontWeight.w600),
                        ),
                        Text(
                          [
                            for (final h in perfil.horarios)
                              if (h.$1 == dia) h.$2,
                          ].join(', '),
                          style: texto(16, cor: cinza),
                        ),
                      ],
                    ),
                  ),
            ],
          ),
        const SizedBox(height: 24),

        // Botão contornado mais pequeno (44 de altura), como no mockup
        OutlinedButton(
          onPressed: onSair,
          style: OutlinedButton.styleFrom(
            foregroundColor: tinta,
            side: const BorderSide(color: tinta, width: 1.5),
            shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(12)),
            minimumSize: const Size.fromHeight(44),
            textStyle: texto(15, peso: FontWeight.w700),
          ),
          child: const Text('Terminar sessão'),
        ),
      ],
    );
```

```
  }
}

// Mostrado quando o pedido à API falha
class ErroPerfil extends StatelessWidget {
  const ErroPerfil({
    super.key,
    required this.mensagem,
    required this.onTentar,
    required this.onSair,
  });

  final String mensagem;
  final VoidCallback onTentar;
  final VoidCallback onSair;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(24),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Text(
            mensagem,
            textAlign: TextAlign.center,
            style: texto(15, cor: cinza),
          ),
          const SizedBox(height: 16),
          Botao(texto: 'Tentar outra vez', onPressed: onTentar),
          const SizedBox(height: 8),
          TextButton(
            onPressed: onSair,
            style: TextButton.styleFrom(foregroundColor: verde),
            child: const Text('Voltar ao Login'),
          ),
        ],
      ),
    );
  }
}
```

O mockup do Perfil tem ainda a localidade e o "Objetivo". A base deste guia não guarda esses dados, por isso o cartão verde mostra o email no lugar da localidade.

Passo 7 — Experimentar

Como há um pacote novo (`google_fonts`), o **R** não chega: no terminal da app carrega em **q** e volta a correr:

```
flutter run -d chrome
```

- [ ] As letras mudam em toda a app: títulos em Bricolage (mais "redondos") e o resto em DM Sans.
- [ ] Onboarding: barra em 3 traços, cartões com a frase de cada desporto, ✓ e fundo lima quando escolhidos, frase "2 desportos escolhidos · a seguir: …" por cima do botão.
- [ ] Continuar fica cinzento até escolher um desporto (e Concluir até escolher um horário).
- [ ] Perfil: "Perfil" com a pílula "Editar", cartão verde-escuro com o avatar lima, "DESPORTOS E NÍVEL" e "DISPONIBILIDADE" em maiúsculas, dias por extenso.
- [ ] Editar → o Passo 1 tem "Voltar", que regressa ao Perfil sem gravar.
- [ ] Criar conta → o Passo 1 não tem "Voltar" (como no mockup).

O que aparece	Porquê	Como resolver
`The argument type 'TextTheme' can't be assigned to the parameter type 'TextTheme?'`	Ficou instalado o `google_fonts` 9.0	`flutter pub add google_fonts:^6.3.3`
`Target of URI doesn't exist: 'package:google_fonts/google_fonts.dart'`	Faltou o Passo 1	`flutter pub add google_fonts:^6.3.3` e voltar a correr
`Undefined name 'tinta'` (ou `cinza`, `linha`…)	O `cores.dart` ainda é o antigo	Fazer o Passo 2
O Perfil continua com o aspeto antigo (letra normal, sem pílula Editar)	O `perfil_screen.dart` ainda é o da Parte 6	Substituir pelo do Passo 6
As letras aparecem normais por uns segundos	O Chrome está a descarregar as letras	É normal na primeira vez